Question 1

Create the SSH banner grabbing script and modify it to capture the banner from FTP port 21.

Approach

Adjust socket connection port number from SSH port 22 to FTP port 21.

Commands

- `python3 script.py` → run python script
- `socket` → create network connection
- `port 21` → FTP service port

Output

STEP 1 : edit script port value

```
┌──(aman㉿kali)-[~]
└─$ nano banner_grabber.py
```

STEP 2 : modify port inside script

```
  GNU nano 8.7.1
port = 21
```

STEP 3 : run script

```
┌──(aman㉿kali)-[~]
└─$ python3 banner_grabber.py
```

Key Observations

Different services run on different ports.
FTP commonly listens on port 21.

What I learned

Port targeting changes reconnaissance results.

Question 2

Modify the banner grabbing script so the user enters the target IP instead of hardcoding it.

Approach

Use input() function to dynamically receive IP address.

Commands

- `input()` → receive user input
- `python3` → execute script

Output

STEP 1 : edit script

```
┌──(aman㉿kali)-[~]
└─$ nano banner_grabber.py
```

STEP 2 : add user prompt

```
  GNU nano 8.7.1
ip = input("Enter target IP: ")
```

STEP 3 : run script

```
┌──(aman㉿kali)-[~]
└─$ python3 banner_grabber.py
Enter target IP: 127.0.0.1
```

Key Observations

Dynamic input increases script flexibility.
Removes need to edit code repeatedly.

What I learned

User input improves tool usability.

Question 3

Modify tcp_server.py so the listening port is provided by the user.

Approach

Prompt for port number and convert input to integer.

Commands

- `input()` → accept user data
- `int()` → convert string to number

Output

STEP 1 : edit tcp server script

```
┌──(aman㉿kali)-[~]
└─$ nano tcp_server.py
```

STEP 2 : add port input

```
  GNU nano 8.7.1
port = int(input("Enter port: "))
```

STEP 3 : run server

```
┌──(aman㉿kali)-[~]
└─$ python3 tcp_server.py
Enter port: 449
```

Key Observations

Ports must be integers for socket binding.
Flexible port selection supports multiple services.

What I learned

Dynamic configuration improves network tool adaptability.

Question 4

Create the FTP password cracking script and modify it to read passwords from a wordlist variable.

Approach

Load password list from file and iterate through entries.

Commands

- `open()` → read file
- `for loop` → iterate passwords
- `python3` → run script

Output

STEP 1 : edit script

```
┌──(aman㉿kali)-[~]
└─$ nano ftp_cracker.py
```

STEP 2 : load wordlist

```
  GNU nano 8.7.1
wordlist = open("/usr/lib/python3/dist-packages/wapitiCore/data/attacks/passwords.txt","r")
for password in wordlist:
        print(password.strip())
```

STEP 3 : run script

```
┌──(aman㉿kali)-[~]
└─$ python3 ftp_cracker.py
admin
administrator
password
guest
demo
PASSWORD
123456
letmein
cisco
info
oracle
manager
test
welcome
Password1
P@ssw0rd
123456789
qwerty
azerty
master
mustang
football
1qaz2wsx
trustno1
hunter
iloveyou
```

Key Observations

Wordlists automate brute force attempts.
Common passwords increase success probability.

What I learned

Automation significantly speeds password testing.

Question 5

Modify banner grabbing script to handle closed ports by printing a message.

Approach

Use try-except block to catch connection errors.

Commands

- `try/except` → error handling
- `socket.error` → detect failure

Output

STEP 1 : edit script

```
┌──(aman㉿kali)-[~]
└─$ nano banner_grabber.py
```

STEP 2 : add exception handling

```
  GNU nano 8.7.1
try:
        banner = s.recv(1024)
except:
        print("no answer")
```

STEP 3 : run script

```
┌──(aman㉿kali)-[~]
└─$ python3 banner_grabber.py
no answer
```

Key Observations

Closed ports trigger connection errors.
Error handling prevents script crashes.

What I learned

Exception handling improves tool reliability.

Security Insights

1. Changing ports enables enumeration of different services.
2. User-supplied IPs allow flexible target selection.
3. Custom port listeners help simulate vulnerable services.
4. Wordlists enable scalable brute force attacks.
5. Error handling avoids detection through crashes.